

Nome: _____

Nº: _____



Laboratório de Algoritmos e Estrutura de Dados

1ª Lista de Atividades – Ordenação Básica

Bubble Sort, Selection Sort and Insertios Sort

- Esta lista de atividades é composta por 13 questões sobre os algoritmos de ordenação básica estudadas em aula;
- Esta lista será considerada como parte da nota da AV2 desta Unidade Curricular;
- A nota máxima nesta atividade é de 10 valores;
- O uso de qualquer recurso de inteligência artificial é vedado;
- As respostas das questões de Código devem ser enviadas para o seu repositório github.

1) Bubble Sort (básico) (0.5 valor)

Implemente `void bubbleSort(int v[], int n)` para ordenar em ordem crescente. Use troca com variável temporária (swap). (Dica: cuidado com os limites do laço interno para não acessar `v[j+1]` fora do vetor.)

2) Bubble Sort instrumentado (comparações e trocas) (0.5 valor)

Implemente Bubble Sort com:

- `comparacoes++` a cada comparação
- `trocas++` a cada swap

Imprima `comparacoes` e `trocas` ao final.

3) Bubble Sort com *early stop* (flag) (0.5 valor)

Crie `void bubbleSortOptimizado(int v[], int n)` usando uma `flag` `houveTroca` para parar quando não houver trocas numa passada. Explique (em comentário no código) por que isso melhora o **melhor caso** para $O(n)$.

4) Comparação experimental: sem otimização vs *early stop* (1 valor)

Gere 3 vetores com $n=20$: (i) já ordenado, (ii) reverso, (iii) aleatório. Rode **Bubble normal** e **Bubble com *early stop*** e compare as métricas. Qual caso reduz mais operações com *early stop*? A resposta dessa questão deve ser gravada em vídeo de até 3 minutos. A sugestão é usar a app Loom como extensão do Google Chrome. O link do vídeo deve ser enviado no Classroom no local indicado.

5) Bubble Sort com parâmetro de ordem (1 valor)

Implemente `void bubbleSortOrdem(int v[], int n, int ordem):`

- `ordem = 1` crescente
- `ordem = -1` decrescente

Teste com: `n=0`, `n=1`, repetidos, mistura de positivos/negativos.

6) Selection Sort (implementação base) (0.5 valor)

Implemente `void selectionSort(int *V, int n)` conforme a ideia de “selecionar o menor e colocar na posição correta”.

7) Traçado (passo a passo) — Selection Sort (0.5 valor)

Para o vetor `[64, 25, 12, 22, 11]`, escreva o estado do vetor ao final de **cada iteração** do laço externo (`i = 0, 1, 2, 3`).

8) Métricas no Selection Sort (1 valor)

Instrumente o Selection Sort com:

- `comparacoes++` a cada comparação entre elementos
- `trocas++` **somente** quando ocorrer troca real (quando `menor != i`)

Execute em (i) ordenado, (ii) reverso, (iii) aleatório, com `n=20`. Compare os resultados e explique por que o número de operações tende a ser “mais constante” que no Bubble.

9) Ordem inicial influencia o Selection Sort? (1 valor)

Com base no seu experimento da questão 8, responda: a ordem inicial do vetor muda significativamente o total de comparações? Justifique citando o comportamento do algoritmo. A resposta dessa questão deve ser gravada em vídeo de até 3 minutos. A sugestão é usar a app Loom como extensão do Google Chrome. O link do vídeo deve ser enviado no Classroom no local indicado.

10) Selection Sort crescente/decrescente (0.5 valor)

Adapte o Selection Sort para receber `ordem` (1 crescente, -1 decrescente). Mostre os testes com o mesmo vetor nas duas ordens e valide a saída.

11) Insertion Sort (implementação base) (1 valor)

Implemente `void insertionSort(int *V, int n)` usando:

- `chave = V[i]`
- deslocamentos enquanto `j > 0 && chave < V[j-1]`
- inserção final da chave na posição correta.

12) Traçado (passo a passo) — Insertion Sort (0.5 valor)

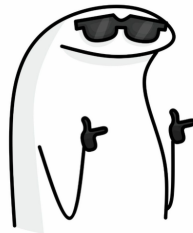
Para o vetor `[5, 3, 4, 1, 2]`, escreva o vetor ao final de cada iteração do `for` (`i=1; i<n; i++`).

13) Insertion Sort instrumentado (comparações e movimentações) (1.5 valor)

Crie uma versão instrumentada que conte:

- `comparacoes`: apenas avaliações da condição do tipo `chave < V[j-1]`
 - `movimentacoes`: atribuições no vetor `V[...] = ...` (inclui deslocamentos e a inserção final da chave)
- Teste com vetores: ordenado, reverso, aleatório ($n=20$). Explique os resultados observados, especialmente no caso “quase ordenado”.

A resposta dessa questão deve ser gravada em vídeo de até 3 minutos. A sugestão é usar a app Loom como extensão do Google Chrome. O link do vídeo deve ser enviado no Classroom no local indicado.



Boa sorte.